

2교시: kube-prometheus-stack 설치

W4D3 Lesson 2 kube-prometheus-stack Helm install

Prometheus Operator 기반 모니터링 스택을 Helm으로 설치하여 클러스터를 관측합니다.

Helm
 Kubernetes Cluster
 관측(Observability) 스택 구축

1 Helm 저장소 추가 (Repository)

prometheus-community

```
$ helm repo add prometheus-community \
https://prometheus-community.github.io/helm-charts
$ helm repo update
```

2 차트 선택 및 확인 (Chart)

kube-prometheus-stack (모니터링 스택)

```
$ helm search repo \
prometheus-community/kube-prometheus-stack --versions
```

3 values 파일 준비/수정 (values.yaml)

```
# values.yaml (예시)
prometheus:
  retention: 35d
grafana:
  adminPassword: "admin"
alertmanager:
  alertmanagerSpec:
    replicas: 2
```

4 Helm 설치/업그레이드 (Install / Upgrade)

릴리스명: monitoring
네임스페이스: monitoring

```
$ helm upgrade --install monitoring \
prometheus-community/kube-prometheus-stack \
-n monitoring --create-namespace -f values.yaml
```

5 설치 네임스페이스

monitoring

- 모든 컴포넌트가 이 네임스페이스에 생성
- RBAC/CRD/서비스/포트 등 자동 구성

설치 확인 (Verification)

Helm 릴리스 확인

```
$ helm list -n monitoring
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
monitoring	monitoring	1	2024-05-20 14:25:18	deployed	kube-prometheus-stack-0.1.0	v0.78.1

주요 컴포넌트 팩드 확인

```
$ kubectl get pods -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE
prometheus-monitoring-kube-p-prometheus-0	2/2	Running	0	3m
alertmanager-monitoring-kube-p-alertmanager-0	2/2	Running	0	3m
grafana-monitoring-grafana-0	1/1	Running	0	3m
prometheus-operator-0f7995d6f-4k9p2	1/1	Running	0	3m
kube-state-metrics-5d770d64c-h82xk	1/1	Running	0	3m
node-exporter-g8kzp	1/1	Running	0	3m
node-exporter-8k2m4	1/1	Running	0	3m
node-exporter-8k2m4	1/1	Running	0	3m

서비스/엔드포인트 확인

```
$ kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
prometheus-operated	ClusterIP	10.96.12.45	<none>	9090/TCP
alertmanager-operated	ClusterIP	10.96.89.221	<none>	9093/TCP
grafana	ClusterIP	10.96.233.15	<none>	30/TCP
kube-state-metrics	ClusterIP	10.96.155.4	<none>	8080/TCP
prometheus-operator	ClusterIP	10.96.200.18	<none>	8080/TCP

참조 정보 (포트 포워딩 예시)

```
kubectl -n monitoring port-forward svc/grafana 3000:80 # http://localhost:3000 (admin/admin)
kubectl -n monitoring port-forward svc/prometheus-operated 9090:9090 # http://localhost:9090
```

6 모니터링 스택 아키텍처 (monitoring 네임스페이스)

데이터 소스 (Metrics)

- 클러스터 컴포넌트: kube-apiserver, kube-scheduler, kube-controller-manager, etcd, CoreDNS
- 노드 메트릭: node-exporter (각 노드)
- 클러스터 상태: kube-state-metrics (클러스터 상태 정보/메타데이터)

서비스 디스커버리 (자동 발견)

- ServiceMonitor (서비스 리소스)
- PodMonitor (포드 리소스)
- Probe (프로브 리소스)
- PrometheusRule (카스텀 리소스)

관측 스택 기반 자동 발견

Prometheus Operator (컨트롤러 플레인)

- prometheus-operator
- Prometheus, Alertmanager 등 모니터링 컴포넌트 생성/관리

플랫폼 구성 요소

- Prometheus: 메트릭 수집/저장, 쿼리 (PromQL), TSDB (시계열 DB)
- Grafana: 대시보드 시각화, 알림 뷰어, 데이터 소스: Prometheus
- Alertmanager: 알림 그룹화/라우팅, 중복 제거, 알림 전송 (Slack, Email 등)

알림 수신 경로

- Slack
- Email
- PagerDuty
- Webhook
- ...

메트릭 수집 흐름 → 자동 발견/관리 → 알림 흐름

CRD (Custom Resource)

7 주요 컴포넌트 설명

컴포넌트	설명
Prometheus Operator	Kubernetes 기반으로 Prometheus, Alertmanager 등을 배포/관리/업데이트/삭제 관리
ServiceMonitor / PodMonitor	특정 라벨을 가진 Service/Pod에서 메트릭을 자동으로 수집하도록 정의
Prometheus	메트릭 수집, 저장, 쿼리 엔진 (PromQL)
Grafana	대시보드 시각화 및 알림 발송
Alertmanager	알림을 그룹화하고 다양한 채널로 전송
node-exporter	노드(CPU, Memory, Disk, Network 등) 메트릭 수집
kube-state-metrics	Kubernetes 리소스 상태(Deployment, Pod 등) 메트릭 생성

8 이미지/아티팩트 소스 (CI → Registry)

소스 Push → GitHub Actions (CI) → Docker Build & Push → Docker Hub (Registry)

예시 태그 전략

버전	태그
1.2.0 (패턴 태그)	latest (최신)
	sha-1b2c3d (커밋 SHA)

9 모범 사례 (Best Practices)

- 관련 리소스: 필요한 Secret만 지정하고 추가적으로 명시
- 리소스 제한: requests/limits 설정
- 알람 룰: 적절한 임계값과 라우팅 정책 구성
- 대시보드 표준화: 팀 공통 대시보드 구성/버전 관리
- 백업: Prometheus 데이터 보존 기간 및 스토리지 관리
- 업그레이드: 장기적으로 스택/차트 버전 업데이트

Tip! 설치 후 빠른 확인: `kubectl get servicemonitors`, `kubectl get prometheusrules`, Grafana 대시보드 Import ID: 315

kube-prometheus-stack으로 클러스터의 상태를 지속적으로 관측하고, 문제를 빠르게 감지해요!

수업 목표

- kube-prometheus-stack 구성요소를 설명한다.
- Helm values file로 Prometheus/Grafana/Alertmanager를 설치한다.
- 설치 후 release, Pod, Service, CRD를 검증한다.

구성요소

구성요소	역할
Prometheus Operator	monitoring CRD reconcile
Prometheus	metric scrape, 저장, query
Grafana	dashboard
Alertmanager	alert routing/silence
node-exporter	node metric
kube-state-metrics	Kubernetes object 상태 metric

values 확인

```
cat week4/day3/labs/kube-prometheus-stack/values.yaml
```

핵심:

```
grafana:
  adminPassword: "paperclip-local"
prometheus:
  prometheusSpec:
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
```

수업용으로 custom ServiceMonitor/PodMonitor/PrometheusRule을 쉽게 읽도록 selector 제한을 풀어둔다.

Helm 설치

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  -f week4/day3/labs/kube-prometheus-stack/values.yaml
```

예상 출력:

```
NAME: kube-prometheus-stack
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
```

kind 검증 기준:

```
kind version
kubectl config current-context
kubectl get nodes -o wide
```

예시:

```
kind v0.32.0
kind-paperclip-w4d3
paperclip-w4d3-control-plane Ready control-plane v1.31.x
```

과정에서는 kind만 기준으로 둔다. 다른 local Kubernetes가 남아 있으면 context, namespace, Service 이름이 섞일 수 있으므로 설치 전 current-context를 반드시 확인한다.

설치 확인

```
helm list -n monitoring
kubectl -n monitoring get pod,svc
kubectl get crd | grep monitoring.coreos.com
```

성공 기준:

```
prometheus-kube-prometheus-stack-prometheus-0 2/2 Running
kube-prometheus-stack-grafana-xxxxx 3/3 Running
alertmanager-kube-prometheus-stack-alertmanager-0 2/2 Running
```

실제 검증 예시:

```
alertmanager-kube-prometheus-stack-alertmanager-0 2/2 Running
kube-prometheus-stack-grafana-... 3/3 Running
kube-prometheus-stack-kube-state-metrics-... 1/1 Running
kube-prometheus-stack-operator-... 1/1 Running
kube-prometheus-stack-prometheus-node-exporter-... 1/1 Running
prometheus-kube-prometheus-stack-prometheus-0 2/2 Running
```

CRD:

```
servicemonitors.monitoring.coreos.com
podmonitors.monitoring.coreos.com
prometheusrules.monitoring.coreos.com
```

Service 이름 확인

chart는 여러 Service를 만든다.

```
kubectl -n monitoring get svc
```

예상:

```
kube-prometheus-stack-grafana      ClusterIP  80/TCP
kube-prometheus-stack-prometheus    ClusterIP  9090/TCP
kube-prometheus-stack-alertmanager  ClusterIP  9093/TCP
```

port-forward 대상은 Service 이름을 정확히 사용한다. 이름이 다르면 `kubectl get svc -n monitoring` 출력으로 현재 cluster의 이름을 먼저 확인한다.

Helm values와 실제 적용 비교

```
helm get values kube-prometheus-stack -n monitoring
helm get manifest kube-prometheus-stack -n monitoring | grep -E "kind:
Prometheus|kind: Grafana|kind: ServiceMonitor" -n
```

확인할 것:

확인	이유
Grafana password	수업 로그인 기준
retention	local disk 사용량 통제
rule selector	custom PrometheusRule 읽기
serviceMonitor selector	custom ServiceMonitor 읽기

values에 썼다고 실제로 적용됐다고 가정하지 않는다. Helm에서는 `helm get values`, Kubernetes에서는 `kubectl get`으로 확인한다.

port-forward

Prometheus:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090
```

Grafana:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

Grafana login:

```
admin / paperclip-local
```

자주 보는 문제

증상	확인
Pod Pending	node resource 부족
Grafana login 실패	values password, secret 확인
CRD 없음	Helm install 실패 또는 chart 렌더링 문제
Prometheus target 없음	ServiceMonitor selector/namespace/port 확인
kind cluster 생성 실패	kind version, Docker Engine version 확인

오래된 kind와 최신 Docker 조합에서는 cluster 생성 중 다음과 같은 오류가 날 수 있다.

```
failed to create cluster: could not find a log line that matches "Reached target
.*Multi-User System.*|detected cgroup v1"
```

이때는 설치 명령 자체가 아니라 kind binary 호환성 문제일 수 있다.

```
go install sigs.k8s.io/kind@latest
~/go/bin/kind version
~/go/bin/kind create cluster --name paperclip-w4d3
```

resource가 부족할 때

kube-prometheus-stack은 가벼운 chart가 아니다. kind node가 작으면 Pending이나 OOM이 생길 수 있다.

```
kubectl -n monitoring get pod
kubectl -n monitoring describe pod <pending-or-crash-pod>
```

대표 메시지:

```
0/1 nodes are available: insufficient memory
OOMKilled
```

해결 방향:

상황	조치
local memory 부족	Docker Desktop/WSL memory 상향
Prometheus OOM	retention/limit 조정
Grafana만 필요	chart values로 일부 component 조정
실습 후 정리	helm uninstall과 namespace 삭제

Evidence Note

```
# W4D3S2 kube-prometheus-stack
- Helm release:
- Prometheus Pod:
- Grafana Pod:
- Alertmanager Pod:
- CRD 확인:
- 접속 URL:
```

한 줄 요약

kube-prometheus-stack은 Prometheus/Grafana만이 아니라 operator, exporter, rule, alert를 묶은 monitoring stack이다.